

\$ system --init db-design

PRODUCT AND CATEGORY MANAGEMENT_SYSTEM

Database Design Document

tags: relational-db · schema · sql · reporting

\$ cat table_of_contents.md

- ▶ 1. Introduction
- ▶ 2. Objectives
- ▶ 3. Entity-Relationship Overview
- ▶ 4. Table Design
- ▶ 5. Primary Key and Foreign Key Relationships
- ▶ 6. SQL Table Creation Scripts
- ▶ 7. Sample Data
- ▶ 8. Product Insertion, Updating, and Deletion
- ▶ 9. Category-wise Product Reports
- ▶ 10. Sample Report Output
- ▶ 11. Conclusion

01 Introduction

The Product and Category Management System is a relational database application designed to organize products under defined categories. The system allows an organization to maintain product information, associate each product with a category, track price and stock levels, and generate category-wise reports for business analysis. This document describes the database design, table structures, key relationships, and the SQL operations required to build and operate the system.

02 Objectives

- ▶ Design normalized tables for the Category and Product entities.
- ▶ Establish primary key and foreign key relationships between the two tables.
- ▶ Store product attributes such as name, category, price, and stock quantity.
- ▶ Support insertion, updating, and deletion of product records.
- ▶ Generate category-wise product reports for analysis and decision-making.

03 Entity-Relationship Overview

The system is built around two core entities: Category and Product. A Category can have many Products, but each Product belongs to exactly one Category. This is a one-to-many relationship, implemented using a foreign key (category_id) in the Product table that references the primary key (category_id) in the Category table.

```
er-diagram.txt

CATEGORY (1) -----< (M) PRODUCT
  category_id (PK)      product_id (PK)
                        category_id (FK -> CATEGORY.category_id)
```

04 Table Design

// 4.1 category table

Stores the list of product categories.

Column Name	Data Type	Constraints	Description
category_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each category
category_name	VARCHAR(100)	NOT NULL, UNIQUE	Name of the category (e.g., Electronics)
description	VARCHAR(255)	NULL	Short description of the category

Column Name	Data Type	Constraints	Description
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation date and time

// 4.2 product table

Stores individual product details, linked to a category.

Column Name	Data Type	Constraints	Description
product_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each product
product_name	VARCHAR(150)	NOT NULL	Name of the product
category_id	INT	FOREIGN KEY -> Category(category_id)	Links product to its category
price	DECIMAL(10,2)	NOT NULL, CHECK (price >= 0)	Selling price of the product
stock	INT	NOT NULL, DEFAULT 0	Quantity available in stock
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation date and time
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Last update date and time

05 Primary Key and Foreign Key Relationships

- ▶ ▶ Category.category_id is the Primary Key of the Category table.
- ▶ ▶ Product.product_id is the Primary Key of the Product table.
- ▶ ▶ Product.category_id is a Foreign Key referencing Category.category_id, enforcing that every product must belong to a valid, existing category.
- ▶ ▶ ON DELETE RESTRICT prevents a category from being deleted while products are still linked to it (can be changed to CASCADE if deleting a category should delete its products).

06 SQL Table Creation Scripts

 create_category.sql

```
CREATE TABLE Category (
  category_id INT AUTO_INCREMENT PRIMARY KEY,
  category_name VARCHAR(100) NOT NULL UNIQUE,
  description VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

● ● ● *create_product.sql*

```
CREATE TABLE Product (  
  product_id INT AUTO_INCREMENT PRIMARY KEY,  
  product_name VARCHAR(150) NOT NULL,  
  category_id INT NOT NULL,  
  price DECIMAL(10,2) NOT NULL CHECK (price >= 0),  
  stock INT NOT NULL DEFAULT 0,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  CONSTRAINT fk_product_category  
    FOREIGN KEY (category_id) REFERENCES Category(category_id)  
    ON DELETE RESTRICT  
    ON UPDATE CASCADE  
);
```

07 Sample Data

// 7.1 insert sample categories

● ● ● *seed_categories.sql*

```
INSERT INTO Category (category_name, description) VALUES  
(  
  'Electronics', 'Electronic gadgets and devices',  
  'Groceries', 'Daily household grocery items',  
  'Stationery', 'Office and school supplies');
```

// 7.2 insert sample products

● ● ● *seed_products.sql*

```
INSERT INTO Product (product_name, category_id, price, stock) VALUES  
(  
  'Wireless Mouse', 1, 599.00, 120),  
  ('Bluetooth Speaker', 1, 1499.00, 45),  
  ('Basmati Rice 5kg', 2, 650.00, 200),  
  ('Notebook Pack (5pc)', 3, 250.00, 300);
```

08 Product Insertion, Updating, and Deletion

// 8.1 insert a new product

● ● ● *insert.sql*

```
INSERT INTO Product (product_name, category_id, price, stock)
```

```
VALUES ('USB-C Charger', 1, 799.00, 80);
```

// 8.2 update a product

● ● ● *update.sql*

```
-- Update price and stock of a product
UPDATE Product
SET price = 549.00, stock = stock + 50
WHERE product_id = 1;
```

// 8.3 delete a product

● ● ● *delete.sql*

```
DELETE FROM Product
WHERE product_id = 4;
```

09 Category-wise Product Reports

// 9.1 list all products under each category

● ● ● *report_all_products.sql*

```
SELECT c.category_name, p.product_name, p.price, p.stock
FROM Category c
JOIN Product p ON c.category_id = p.category_id
ORDER BY c.category_name, p.product_name;
```

// 9.2 category-wise product count and total stock value

● ● ● *report_stock_value.sql*

```
SELECT c.category_name,
       COUNT(p.product_id) AS total_products,
       SUM(p.stock) AS total_stock,
       SUM(p.price * p.stock) AS total_stock_value
FROM Category c
LEFT JOIN Product p ON c.category_id = p.category_id
GROUP BY c.category_name
ORDER BY total_stock_value DESC;
```

// 9.3 low stock report (stock below threshold) by category

● ● ● *report_low_stock.sql*

```
SELECT c.category_name, p.product_name, p.stock
FROM Category c
JOIN Product p ON c.category_id = p.category_id
WHERE p.stock < 50
ORDER BY c.category_name;
```

10 Sample Report Output

Category	Product	Price (Rs.)	Stock
Electronics	Wireless Mouse	549.00	170
Electronics	Bluetooth Speaker	1499.00	45
Groceries	Basmati Rice 5kg	650.00	200

11 Conclusion

This design provides a normalized, referentially-consistent structure for managing categories and products. The one-to-many relationship between Category and Product ensures data integrity, while the provided SQL scripts cover schema creation, CRUD operations, and category-wise reporting needed for day-to-day inventory and catalog management.

\$ process exited (code 0)